NAMA : Aryo manubulu

NIM : 257111065

KELAS/PRODI : C/ PENDIDIKAN INFORMATIKA

MATA KULIAH : STRUKTUR DATA

```
class Mahasiswa:
    def __init__(self, nim, nama, jurusan):
        self.nim = nim
        self.nama = nama
        self.jurusan = jurusan

class Node:
    def __init__(self, data):
        self.data = data
        self.next = None
        self.prev = None # Hapus baris ini kalau mau
jadi Single Linked List

class LinkedListMahasiswa:
    def __init__(self, double=True):
        self.head = None
        self.tail = None
        self.double = double # True = Double, False =
Single

    # 1. TAMBAH DI AWAL
    def tambah_awal(self, nim, nama, jurusan):
        mhs = Mahasiswa(nim, nama, jurusan)
        node = Node(mhs)
        if not self.head:
            self.head = self.tail = node
        else:
            node.next = self.head
            if self.double:
                self.head.prev = node
            self.head = node
        print(f"-> {nama} ditambah di AWAL")

    # 2. TAMBAH DI AKHIR
    def tambah_akhir(self, nim, nama, jurusan):
        mhs = Mahasiswa(nim, nama, jurusan)
        node = Node(mhs)
        if not self.head:
            self.head = self.tail = node
        else:
```

```
            self.tail.next = node
            if self.double:
                node.prev = self.tail
            self.tail = node
        print(f"-> {nama} ditambah di AKHIR")

    # 3. TAMBAH DI TENGAH (URUT NIM)
    def tambah_urut_nim(self, nim, nama, jurusan):
        mhs = Mahasiswa(nim, nama, jurusan)
        node = Node(mhs)

        if not self.head or nim < self.head.data.nim:
            self.tambah_awal(nim, nama, jurusan)
            return

        curr = self.head
        while curr.next and curr.next.data.nim < nim:
            curr = curr.next

        # Cek duplikat NIM
        if curr.data.nim == nim:
            print(f"!! NIM {nim} sudah ada!")
            return

        node.next = curr.next
        curr.next = node
        if self.double:
            if node.next:
                node.next.prev = node
            node.prev = curr
            if curr == self.tail:
                self.tail = node
        print(f"-> {nama} ditambah urut NIM")

    # 4. HAPUS BERDASARKAN NIM
    def hapus(self, nim):
        if not self.head:
            print("List kosong!")
            return

        curr = self.head
        while curr and curr.data.nim != nim:
            curr = curr.next

        if not curr:
            print(f"NIM {nim} tidak ditemukan!")
            return
```

```
        if curr == self.head:
            self.head = curr.next
            if self.head and self.double:
                self.head.prev = None
            if curr == self.tail:
                self.tail = None
        elif curr == self.tail:
            self.tail = curr.prev if self.double else
self._get_prev(curr)
            if self.tail:
                self.tail.next = None
        else:
            prev_node = curr.prev if self.double else
self._get_prev(curr)
            prev_node.next = curr.next
            if self.double and curr.next:
                curr.next.prev = prev_node

        print(f"-> Data NIM {nim} berhasil dihapus")

    def _get_prev(self, node_target): # Helper khusus
Single Linked List
        curr = self.head
        while curr and curr.next != node_target:
            curr = curr.next
        return curr

    # 5. CARI BERDASARKAN NIM (Linear Search)
    def cari(self, nim):
        curr = self.head
        posisi = 0
        while curr:
            if curr.data.nim == nim:
                print(f"DITEMUKAN di posisi {posisi}:
{curr.data.nim} | {curr.data.nama} |
{curr.data.jurusan}")
                return curr.data
            curr = curr.next
            posisi += 1
        print(f"NIM {nim} tidak ditemukan")
        return None

    # 6. TAMPILKAN SEMUA
    def tampilkan(self):
        if not self.head:
            print("Belum ada data mahasiswa")
            return
        print("\n--- DAFTAR MAHASISWA ---")
```

```
        print(f"{'NIM':<10} {'Nama':<15} {'Jurusan'}")
        print("-" * 40)
        curr = self.head
        while curr:
            print(f"{curr.data.nim:<10}
{curr.data.nama:<15} {curr.data.jurusan}")
            curr = curr.next
        print("-" * 40 + f" [Mode: {'DOUBLE' if
self.double else 'SINGLE'} Linked List]\n")

# --- PROGRAM UTAMA (MENU) ---
if __name__ == "__main__":
    print("Pilih implementasi:")
    print("1. Single Linked List")
    print("2. Double Linked List")
    pilih = input("Pilih (1/2): ")
    is_double = True if pilih == "2" else False

    ll = LinkedListMahasiswa(double=is_double)

    # DATA AWAL OTOMATIS
    data_awal = [
        (257111067, "arto", "keperawatan"),
        (257111083, "Inaya", "pendikan informatika"),
        (257111079, "Aldo", "farmasi"),
        (257111080, "yosua", "manajemen  ")

    ]
    for nim, nama, jurusan in data_awal:
        ll.tambah_urut_nim(nim, nama, jurusan)

    print("\n[3 Data awal berhasil ditambahkan, silahkan
pilih menu 6 untuk tampilkan]")

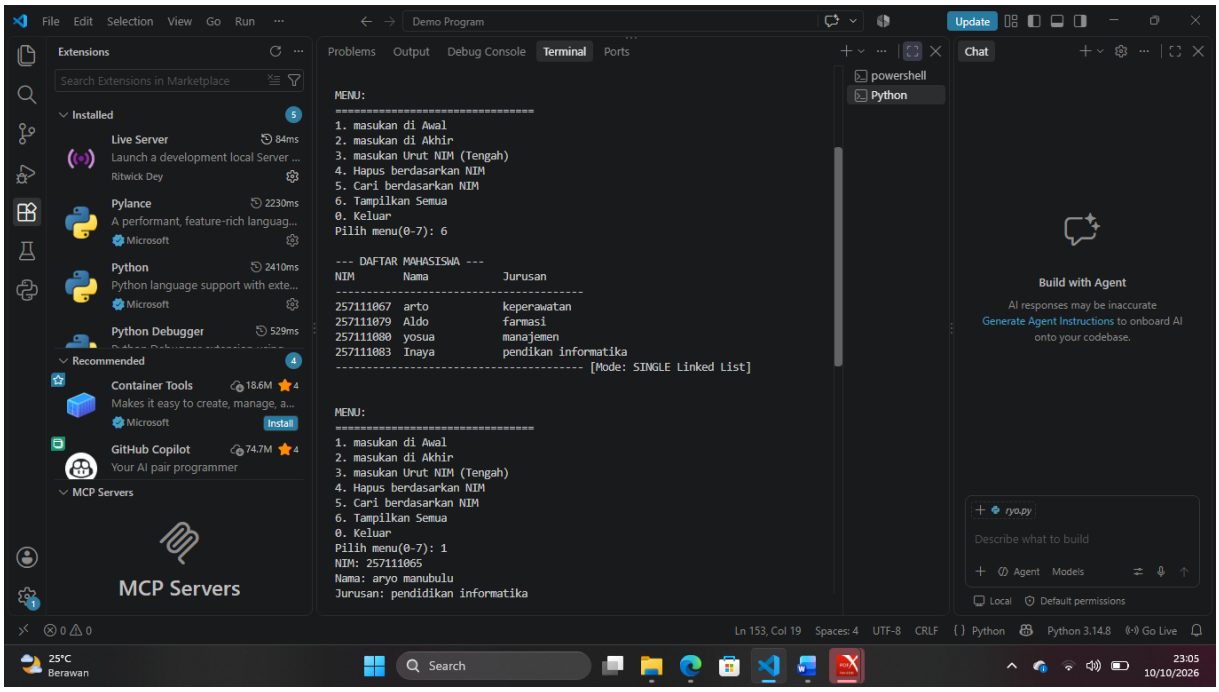
    while True:
        print("\nMENU:")
        print("================================")
        print("1. masukan di Awal")
        print("2. masukan di Akhir")
        print("3. masukan Urut NIM (Tengah)")
        print("4. Hapus berdasarkan NIM")
        print("5. Cari berdasarkan NIM")
        print("6. Tampilkan Semua")
        print("0. Keluar")

        menu = input("Pilih menu(0-7): ")

        if menu == "0":
```

```
            break
        elif menu in ["1","2","3"]:
            nim = int(input("NIM: "))
            nama = input("Nama: ")
            jurusan = input("Jurusan: ")
            if menu == "1": ll.tambah_awal(nim, nama,
jurusan)
            elif menu == "2": ll.tambah_akhir(nim, nama,
jurusan)
            else: ll.tambah_urut_nim(nim, nama, jurusan)
        elif menu == "4":
            nim = int(input("NIM yang mau dihapus: "))
            ll.hapus(nim)
        elif menu == "5":
            nim = int(input("NIM yang dicari: "))
            ll.cari(nim)
        elif menu == "6":
            ll.tampilkan()
```

HASILNYA



Analisis Kompleksitas Single Linked List

1. Pengertian Single Linked List

Single Linked List adalah struktur data yang terdiri dari kumpulan node. Setiap node menyimpan data mahasiswa dan satu pointer `next` yang menunjuk ke node berikutnya.

Pada program kamu, setiap node menyimpan NIM, nama, dan jurusan mahasiswa.

2. Analisis kompleksitas waktu (Time Complexity)

Kompleksitas waktu menunjukkan seberapa banyak operasi yang dibutuhkan ketika jumlah data mahasiswa bertambah. Misalkan adalah jumlah mahasiswa dalam linked list.

Operasi pada program	**Kompleksitas**	**Penjelasan**
Tambah di awal (tambah_awal)	O(1)	Langsung mengubah head.
Tambah di akhir (tambah_akhir)	O(1)	Langsung menggunakan tail yang sudah tersedia
Tambah urut NIM (tambah_urut_nim)	O(n)	Harus mencari posisi berdasarkan NIM.
Hapus berdasarkan NIM (hapus)	O(n)	Harus mencari node yang akan dihapus.
Cari berdasarkan NIM (cari)	O(n)	Memeriksa node satu per satu.
Tampilkan semua (tampilkan)	O(n)	Mengunjungi seluruh node.
Mencari node sebelumnya (_get_prev)	O(n)	Menelusuri list dari awal.

3. Analisis kompleksitas ruang (Space Complexity)

Kompleksitas ruang menunjukkan jumlah memori yang dibutuhkan oleh program.

- Setiap mahasiswa disimpan dalam satu objek `Mahasiswa`.
- Setiap mahasiswa memiliki satu node yang menyimpan data dan pointer `next`.
- Jika menggunakan mode Single Linked List, pointer `prev` sebenarnya tidak dibutuhkan secara fungsional, meskipun pada kode kamu atribut tersebut masih dibuat dengan nilai `None`.

Dengan demikian, kompleksitas ruang untuk menyimpan mahasiswa adalah: {O(n)

Artinya, semakin banyak data mahasiswa, semakin banyak memori yang digunakan.

4. Kesimpulan

Single Linked List cocok digunakan untuk pengelolaan data mahasiswa karena penambahan data di awal dan akhir dapat dilakukan dengan cepat. Namun, pencarian, penghapusan berdasarkan NIM, dan penambahan data secara terurut membutuhkan waktu yang dapat meningkat seiring bertambahnya jumlah mahasiswa.

Secara umum, program kamu memiliki kompleksitas ruang , dengan operasi utama yang berkisar antara dan .